

Module 6 Week A — Integration Task: Entity Analysis

This integration task brings together everything from Week A – text preprocessing, spaCy pipelines, and Named Entity Recognition – into a complete entity analysis pipeline across a full corpus. You will preprocess the climate articles dataset, run NER at scale, aggregate entity statistics, produce a visualization, and write an analytical report.

This pipeline produces the entity extraction component that Week B's integration will extend with semantic embeddings to create the module's culminating deliverable.

Due: Thursday end of day. Resubmissions are accepted through Saturday of the assignment week.

Learning Objectives

- Preprocess a full multilingual corpus with language-aware handling
- Run NER across an entire dataset using the system you evaluated in the lab
- Aggregate entity statistics: frequency, co-occurrence, and cross-category patterns
- Produce a publication-quality entity frequency visualization
- Write an analytical report connecting entity patterns to domain insights

Setup

1. **Accept the assignment**

2. **Clone your repo:**

```
git clone <your-repo-url>
cd <your-repo-name>
```

3. **Install dependencies:**

```
pip install -r requirements.txt
```

4. **Download the spaCy English model** (if not already downloaded):

```
python -m spacy download en_core_web_sm
```

5. **Create a working branch:**

```
git checkout -b integration-6a-entity-analysis
```

The starter file `entity_analysis.py` contains function signatures with docstrings and TODO comments. The dataset is at `data/climate_articles.csv` – the same dataset you used in the lab. Implement each function following the TODO directions, the task descriptions below, and the Week A reading.

The `nlp` object is loaded **once** in `_main_` and passed into the function that needs it – the same dependency-injection pattern you used in the lab.

Task 1: Load and Preprocess the Full Corpus

Implement two functions:

`load_corpus(filepath)` – read `data/climate_articles.csv` into a `DataFrame` and return it unchanged.

`preprocess_corpus(df)` – return a copy of the corpus with a new `processed_text` column:

- Apply **Unicode NFC normalization** to every row's `text`. NFC collapses visually identical characters that have multiple byte representations (e.g., pre-composed “e” U+00E9 vs. decomposed “e” + combining acute U+0301) so later string comparisons and aggregations work consistently.
- For **English** rows (`language == 'en'`), store the NFC-normalized text in `processed_text`.
- For **Arabic** rows (`language == 'ar'`), do not attempt English NLP processing. Either pass the NFC-normalized text through unchanged or store an empty string. Either choice is fine – what matters is that the pipeline does not crash.
- Leave the original `text` column intact. NER needs the raw, capitalized, punctuated text to work correctly – do not lowercase, strip punctuation, or lemmatize before NER. Those transformations destroy the signals the NER model uses to recognize entities.

Return the updated `DataFrame`.

Why this is different from the lab's `preprocess_text`. In the lab, preprocessing was for analysis of tokens (lemmas, stop word removal). Here, preprocessing is for language-aware pipeline setup – tagging what each row is so the right downstream steps run on it. The aggressive normalization from the lab would break NER.

Task 2: Run NER on the Full Corpus

Implement `run_ner_pipeline(df, nlp)`:

- Filter the `DataFrame` to English-language rows (`language == 'en'`)
- Run spaCy NER on every English text's raw `text` column using the injected `nlp` pipeline. `en_core_web_sm` is the production-appropriate tool for corpus-level extraction – fast, CPU-native, designed for batch processing
- Extract all entities into a structured `DataFrame` with columns: `text_id`, `entity_text`, `entity_label`, `start_char`, `end_char`
- Return the entity `DataFrame`

Task 3: Aggregate Entity Statistics

Implement `aggregate_entity_stats(entity_df, articles_df)`:

- Top entities:** compute the top 20 most frequent entities (columns: `entity_text`, `entity_label`, `count`)
- Label counts:** total count per entity label (dict of `entity_label` -> int)
- Co-occurrence:** for each pair of entities that appear in the same text, count how many texts contain both (`DataFrame` with columns `entity_a`, `entity_b`, `co_count`). With ~130 English texts the raw pair list can be very large — cap the result at the **top 50 pairs by `co_count`** (or filter to pairs with `co_count >= 2`) so downstream consumers and the report stay readable.
- Per-category:** join `entity_df` against `articles_df` on `text_id = id` to attach the article `category` to each entity, then count `(category, entity_label)` combinations. Returns a `DataFrame` with columns `category`, `entity_label`, `count` – this is what lets you say “policy articles have 3x more ORG entities than impact articles.”

Return a dictionary with keys `top_entities`, `label_counts`, `co_occurrence`, `per_category`. Print a summary of the key statistics to the console.

Task 4: Visualize Entity Distribution

Implement `visualize_entity_distribution(stats, output_path="entity_distribution.png")`:

- Take the `stats` dict returned by `aggregate_entity_stats` (read `stats["top_entities"]`)
- Create a bar chart showing the top 20 most frequent entities, colored or grouped by entity type
- Label the axes clearly (entity names on one axis, frequency on the other)
- Include a legend showing entity type colors
- Save the chart to `output_path`

The visualization should be readable – if entity names overlap, use a horizontal bar chart or rotate labels.

Task 5: Generate the Entity Report

This task has **two parts**:

Part A – implement `generate_report(stats, co_occurrence)`. Return a formatted string summarizing:

- Entity counts per type (from `stats["label_counts"]`)
- Top 5 most frequent entities (from `stats["top_entities"]`)
- Top 3 co-occurring entity pairs (from `co_occurrence`)
- A one-paragraph summary of what the numbers show

The autograder checks that the function returns a non-empty string. The TA reads it for structure and accuracy.

Part B – write the analytical narrative in your PR description. Use your aggregation, per-category breakdown, and visualization to answer:

- Key findings** – What are the most prominent entities in the corpus? Which entity types dominate?
- Patterns across categories** – How does entity distribution differ between policy, science, impact, and adaptation texts? Are certain organizations or locations associated with specific categories? (Use the `per_category` breakdown here.)
- Co-occurrence insights** – Which entities frequently appear together? What does this suggest about the relationships between organizations, locations, and topics in climate discourse?
- Dashboard recommendation** – Propose one specific feature for a climate research dashboard based on your entity analysis. What would the feature do? What entities or patterns would it surface? Why would it be useful for a climate researcher?

Challenge Extensions

These are optional extensions for learners who want to go deeper. They are not graded and do not affect your integration score. No starter code is provided – design and implement these independently.

Tier 1: Temporal Entity Analysis

The corpus contains DATE entities and articles from different time periods.

- Extract and normalize all DATE entities to a standard format (year or year-month where possible).
- Analyze how entity mentions change over time: are certain organizations (IPCC, COP) mentioned more in recent texts? Do location mentions shift?
- Produce a timeline visualization showing entity frequency trends.
- Write a 1-paragraph analysis of temporal patterns in climate entity coverage.

Tier 2: Entity Relationship Graph

Build a structured entity relationship representation:

- Define entity relationships from co-occurrence: if two entities appear in the same sentence (not just the same text), they have a stronger relationship than if they co-occur only at the document level.
- Implement sentence-level and document-level co-occurrence separately using spaCy's sentence segmentation.
- Build an adjacency matrix of entity relationships weighted by co-occurrence strength.
- Export the graph as a CSV edge list with columns: `entity_1`, `entity_2`, `weight`, `relationship_level` (sentence vs. document).
- Visualize the top 30 entity relationships as a network graph with edge thickness proportional to co-occurrence strength.

Tier 3: Domain-Adapted NER with EntityRuler

spaCy's built-in NER misses domain-specific entities like “COP28”, “Paris Agreement”, “IPCC AR6”, and “2-degree target” because they were not in the training data.

- Build a spaCy `EntityRuler` with at least 15 custom entity patterns covering climate-specific terminology. Define custom entity labels (e.g., `CLIMATE_EVENT`, `POLICY`, `REPORT`, `THRESHOLD`) that extend the standard NER schema.
- Integrate the EntityRuler into the spaCy pipeline (before or after the built-in NER – experiment with both positions and document the difference).
- Re-run the full corpus analysis with the augmented pipeline. Compare entity counts and types before and after adding custom rules.
- Evaluate the augmented pipeline on the gold standard. Does precision improve? Does recall? Compute the delta for each metric.
- Write a 2-paragraph analysis: when do custom rules help, and when do they introduce noise?

Submission

Open a pull request from `integration-6a-entity-analysis` to `main`. Your PR description must include:

- Entity statistics summary (top entities by type, counts)
- Key findings and patterns across categories
- Dashboard recommendation (1 paragraph)
- Paste your PR URL into TalentLMS -> Module 6 Week A -> Integration 6A to submit this assignment.

